Wallet Service

RecargaPay – Technical Assessment

Java 25 · Spring Boot 4 · PostgreSQL 17 · Redis 7 · Docker

Project Overview

Core Purpose

Mission-critical REST API for digital wallets: deposit, withdraw, transfer, current balance and historical balance.

Stack

Java 25 · Spring Boot 4 · PostgreSQL 17 · Redis 7 · Flyway · Testcontainers · Springdoc OpenAPI 3.

Architecture

Modular monolith with explicit wallet ↔ transaction boundary, ready to be extracted to microservices.

Architecture — Modular Monolith

Wallet Module

- WalletController
- WalletService
- WalletRepository
- Wallet (entity)
- Request/Response DTOs

Transaction Module

- TransactionController
- TransactionService
- WalletComponent ⬡
- TransactionRepository
- Transaction (entity)

Shared Module

- DistributedLockService
- CacheConfig
- GlobalExceptionHandler
- Enums / Exceptions

⬡ WalletComponent = integration proxy. Replaced by Feign/RestClient when extracting to microservices — no changes to TransactionService.

Distributed Lock with Redis (DA-02 · DA-03)

Acquisition Flow

1. SET wallet:lock:{id} {uuid} NX PX 10000
   → NX: atomic, only sets if key does not exist
   → PX 10s: TTL protects against crash

2. If locked → retry every 50ms for up to 5s
   → Timeout: LockAcquisitionException → HTTP 409

3. Execute operation inside the lock

4. Release via atomic Lua script:
   if get(key)==uuid then del(key) end
   → Only the lock owner can release it

DA-03 — Release after TX Commit

❌ **Without DA-03:**
lock → modify → release → [TX pending] → another thread reads stale balance

✅ **With DA-03:**
lock → modify → TX commits → release → another thread reads correct balance

Deadlock Prevention

Concurrent A→B and B→A transfers deadlock if locks are acquired out of order.

Solution: always acquire locks in ascending UUID order.

```
sortedIds = [sourceId, destId].stream().sorted().toList()
```

Data Model — PostgreSQL

tb_wallet

- id UUID (PK)
- user_id UUID UNIQUE
- balance DECIMAL(19,4)
- currency VARCHAR(3) 'BRL'
- created_at TIMESTAMPTZ
- updated_at TIMESTAMPTZ
- ——— CHECK (balance >= 0)

tb_transaction

- id UUID (PK)
- wallet_id UUID (FK)
- type DEPOSIT | WITHDRAWAL
- TRANSFER_IN | TRANSFER_OUT
- amount DECIMAL(19,4)
- balance_after DECIMAL(19,4)
- reference_id UUID (nullable)
- description VARCHAR(255)
- created_at TIMESTAMPTZ
- ——— CHECK (amount > 0)
- ——— CHECK (balance_after >= 0)

reference_id: TRANSFER_OUT.id ↔ TRANSFER_IN.id — bidirectional link between the two records of a transfer.

Historical Balance & Redis Cache (DA-04)

Historical Balance — O(log n)

DA-04: balance_after denormalized on every transaction eliminates replay.

Query:

```
SELECT balance_after FROM tb_transaction
WHERE wallet_id = :id
  AND created_at <= :ts
ORDER BY created_at DESC
LIMIT 1
```

Composite index (wallet_id, created_at) guarantees O(log n).
If no transaction exists before the timestamp → returns zero balance.

Redis Cache — Wallet Entity

Cache name	wallets
Key	wallets::{walletId}
TTL	10 minutes
Serialization	JSON + @class (polymorphic)
Null values	disabled
@Cacheable	findById (read)
@CachePut	after any write
@CacheEvict	after transaction operation

API Design — Endpoints REST

POST	/api/v1/wallets	Create new wallet
GET	/api/v1/wallets/{walletId}	Return wallet details
GET	/api/v1/wallets/{walletId}/balance	Current balance
GET	/api/v1/wallets/{walletId}/balance/history?at=	Balance at a point in time
POST	/api/v1/wallets/{walletId}/deposit	Deposit funds
POST	/api/v1/wallets/{walletId}/withdraw	Withdraw funds
POST	/api/v1/wallets/transfer	Transfer (atomic, anti-deadlock)
GET	/api/v1/wallets/{walletId}/transactions	Paginated transaction history
GET	/actuator/health	Health check

Error Handling — RFC 7807

404	**`WalletNotFoundException`** Wallet not found	*ProblemDetail*
409	**`WalletAlreadyExistsException`** User already has a wallet	*ProblemDetail*
409	**`LockAcquisitionException`** System under high load — lock not acquired	*ProblemDetail*
422	**`InsufficientFundsException`** Insufficient funds for withdrawal/transfer	*ProblemDetail*
400	**`InvalidRequestException`** Business rule violation (e.g. transfer to self)	*ProblemDetail*
400	**`MethodArgumentNotValidException`** Bean Validation — per-field error map	*ProblemDetail*
500	**`Exception (fallback)`** Unexpected error	*ProblemDetail*

All error responses follow RFC 7807 (ProblemDetail) — type, title, status, detail, instance.

Testing Strategy

Unit Tests ~60%

Service logic · Edge cases · Mockito

Integration ~30%

Testcontainers · Real PostgreSQL · Real Redis · TX behavior

E2E / API ~10%

Full Spring Boot Test · MockMvc · OpenAPI contract

Key scenarios:

wallet creation · duplicate · insufficient funds · transfer to self · deadlock prevention · historical balance (past timestamp) · paginated history

Environment Configuration & Deploy

Profile	Database	Redis	Hikari pool	Log
local	localhost:5432	localhost:6379	default	DEBUG
dev	env vars	env vars	default	DEBUG
stage	env vars	env vars	5–10	INFO
production	env vars	env vars	10–20	WARN

Quick Start

```
docker-compose up -d
mvn spring-boot:run
# Swagger UI → http://localhost:8080/swagger-ui.html
# Redoc   → http://localhost:8080/redoc.html
```

Architectural Decisions & Trade-offs

DA-01 **Modular monolith**

WalletComponent isolates the boundary — extractable to microservices without changing TransactionService.

DA-02 **Redis lock vs. DB lock**

SET NX PX — works in any topology. SELECT FOR UPDATE breaks in microservices.

DA-03 **Release after commit**

TransactionSynchronization.afterCompletion eliminates the read-after-write inconsistency window.

DA-04 **balance_after denorm.**

O(log n) history lookup vs O(n) replay. Trade-off: extra storage per transaction record.

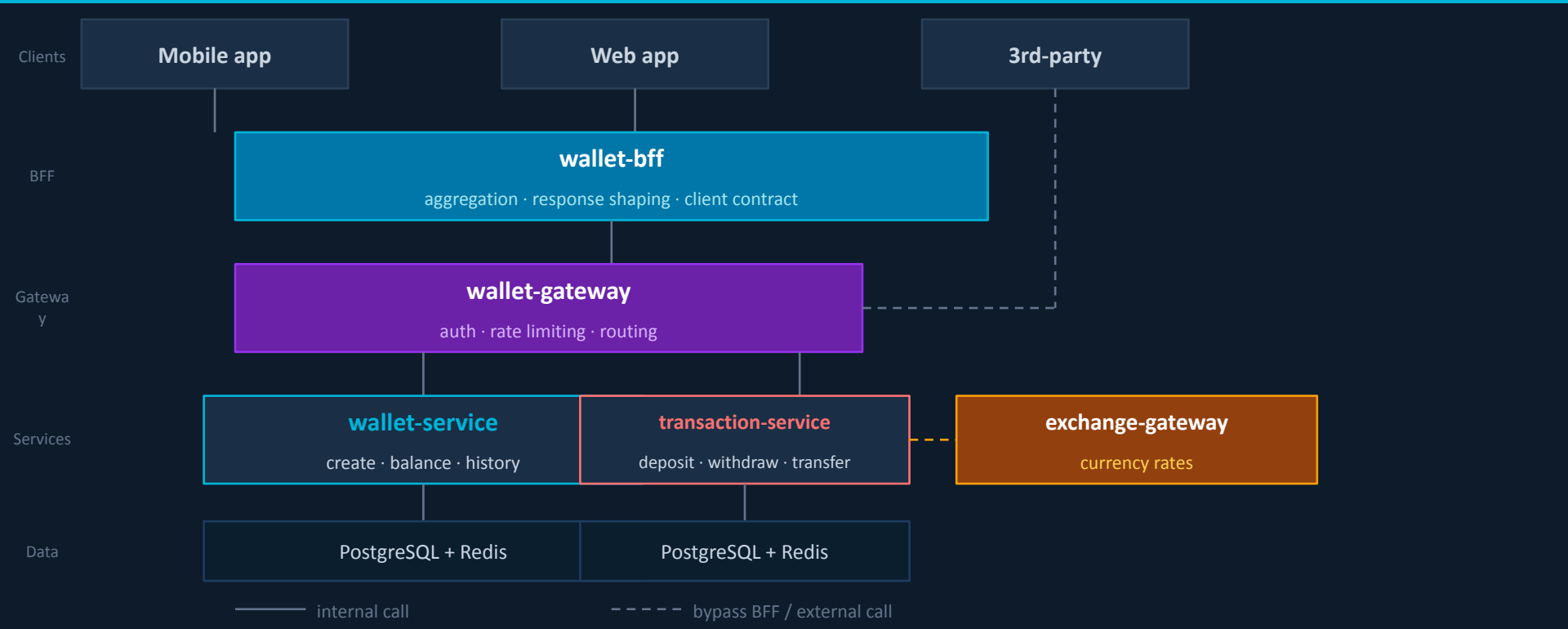
DA-05 **DB constraints**

CHECK constraints in schema as a second line of defence — guards against bugs and direct DB access.

Scope **No auth / rate limit**

OAuth2/JWT and sliding-window rate limiting are production steps — out of scope for this assessment.

Microservices — Proposed Architecture



Ready to Build

Java 25 · Spring Boot 4 · Redis locks · Cache · Tests with Testcontainers · ~7h

Modular Monolith	Distributed Redis Lock	Cache Redis	Historical O(log n)	RFC 7807 Errors